

# Deep Learning

**A Napkin-style companion for campus placements:  
from the perceptron to Transformers and LLMs, with derivations,  
hand numerics, traps, and code that checks itself**

Philospher

Built October 6, 2026



# Preface

[STUB: preface] Versions: 2.14.1+cu130.



# Contents

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>Preface</b>                                                     | <b>iii</b> |
| <b>I Foundations</b>                                               | <b>7</b>   |
| <b>1 Perceptron and the multilayer perceptron</b>                  | <b>9</b>   |
| 1.1 The artificial neuron . . . . .                                | 9          |
| 1.2 The perceptron . . . . .                                       | 10         |
| 1.3 XOR: what one neuron cannot do . . . . .                       | 12         |
| 1.4 The multilayer perceptron . . . . .                            | 14         |
| 1.5 Universal approximation, and why depth still matters . . . . . | 15         |
| 1.6 Capacity: what makes a network more expressive . . . . .       | 17         |
| 1.7 A few problems to think about . . . . .                        | 18         |
| <b>2 Activations, losses and output layers</b>                     | <b>19</b>  |
| 2.1 Activation functions . . . . .                                 | 19         |
| 2.2 Output layers: sigmoid and softmax . . . . .                   | 21         |
| 2.3 Loss functions . . . . .                                       | 23         |
| 2.4 A few problems to think about . . . . .                        | 26         |
| <b>3 Backpropagation</b>                                           | <b>27</b>  |
| 3.1 Backpropagation . . . . .                                      | 27         |
| <b>4 Optimisers and learning-rate schedules</b>                    | <b>29</b>  |
| 4.1 Optimisers and learning-rate schedules . . . . .               | 29         |
| <b>5 Initialisation, normalisation and regularisation</b>          | <b>31</b>  |
| 5.1 Initialisation, normalisation and regularisation . . . . .     | 31         |
| <b>II Architectures</b>                                            | <b>33</b>  |
| <b>6 Convolutional networks</b>                                    | <b>35</b>  |
| 6.1 Convolutional networks . . . . .                               | 35         |
| <b>7 Recurrent networks</b>                                        | <b>37</b>  |
| 7.1 Recurrent networks . . . . .                                   | 37         |
| <b>8 Attention and the Transformer</b>                             | <b>39</b>  |
| 8.1 Attention and the Transformer . . . . .                        | 39         |
| <b>III Modern Deep Learning</b>                                    | <b>41</b>  |
| <b>9 Language models and LLMs</b>                                  | <b>43</b>  |
| 9.1 Language models and LLMs . . . . .                             | 43         |
| <b>10 Generative and representation learning</b>                   | <b>45</b>  |
| 10.1 Generative and representation learning . . . . .              | 45         |

|           |                                |           |
|-----------|--------------------------------|-----------|
| <b>IV</b> | <b>Interview Mode</b>          | <b>47</b> |
| <b>11</b> | <b>Practice and PyTorch</b>    | <b>49</b> |
| 11.1      | Practice and PyTorch . . . . . | 49        |
| <b>12</b> | <b>Implement it</b>            | <b>51</b> |
| 12.1      | Implement it . . . . .         | 51        |
| <b>13</b> | <b>Rapid-fire</b>              | <b>53</b> |
| 13.1      | Rapid-fire . . . . .           | 53        |
| <b>14</b> | <b>Problems</b>                | <b>55</b> |
| 14.1      | Problems . . . . .             | 55        |
| <b>V</b>  | <b>Backmatter</b>              | <b>57</b> |
| <b>A</b>  | <b>Hints to problems</b>       | <b>59</b> |
| <b>B</b>  | <b>Solutions to problems</b>   | <b>61</b> |
|           | <b>Bibliography</b>            | <b>63</b> |

# I

## Foundations

# Part I: Contents

---

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Perceptron and the multilayer perceptron</b>                | <b>9</b>  |
| 1.1      | The artificial neuron . . . . .                                | 9         |
| 1.2      | The perceptron . . . . .                                       | 10        |
| 1.3      | XOR: what one neuron cannot do . . . . .                       | 12        |
| 1.4      | The multilayer perceptron . . . . .                            | 14        |
| 1.5      | Universal approximation, and why depth still matters . . . . . | 15        |
| 1.6      | Capacity: what makes a network more expressive . . . . .       | 17        |
| 1.7      | A few problems to think about . . . . .                        | 18        |
| <b>2</b> | <b>Activations, losses and output layers</b>                   | <b>19</b> |
| 2.1      | Activation functions . . . . .                                 | 19        |
| 2.2      | Output layers: sigmoid and softmax . . . . .                   | 21        |
| 2.3      | Loss functions . . . . .                                       | 23        |
| 2.4      | A few problems to think about . . . . .                        | 26        |
| <b>3</b> | <b>Backpropagation</b>                                         | <b>27</b> |
| 3.1      | Backpropagation . . . . .                                      | 27        |
| <b>4</b> | <b>Optimisers and learning-rate schedules</b>                  | <b>29</b> |
| 4.1      | Optimisers and learning-rate schedules . . . . .               | 29        |
| <b>5</b> | <b>Initialisation, normalisation and regularisation</b>        | <b>31</b> |
| 5.1      | Initialisation, normalisation and regularisation . . . . .     | 31        |

---

# 1 Perceptron and the multilayer perceptron

Every network in this book, from a two-layer classifier to a Transformer with a hundred layers, is built from one object: a weighted sum followed by a fixed non-linear function. This chapter takes that object apart, shows exactly what one of them can and cannot do (the XOR story that OAs keep asking about), and then stacks them into the multilayer perceptron whose parameters we learn to count in our sleep.

## §1.1 The artificial neuron

*Prototypical example for this section:* Inputs  $\mathbf{x} = (0.5, -1, 2)$ , weights  $\mathbf{w} = (0.4, 0.3, -0.2)$ , bias  $b = 0.1$ : the neuron outputs  $\phi(\mathbf{w}^\top \mathbf{x} + b)$  for some transfer function  $\phi$ .

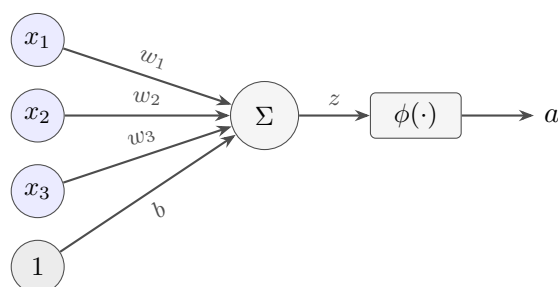
### §1.1.i Definition

**Definition 1.1.1.** An **artificial neuron** with weights  $\mathbf{w} \in \mathbb{R}^d$ , bias  $b \in \mathbb{R}$  and **activation** (or **transfer**) function  $\phi : \mathbb{R} \rightarrow \mathbb{R}$  maps an input  $\mathbf{x} \in \mathbb{R}^d$  to

$$a = \phi(z), \quad z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b.$$

The number  $z$  is the **pre-activation** (or weighted sum, or logit);  $a$  is the **activation**.

The classical choices of  $\phi$  are the **step** (Heaviside) function  $\mathbb{K}[z \geq 0]$ , the **sigmoid**  $\sigma(z) = 1/(1 + e^{-z})$ ,  $\tanh z$  and the **ReLU**  $\max(0, z)$ . **Chapter 2** compares them properly; for now they are just four ways to squash or clip  $z$ .



### Example 1.1.2 (One neuron, four transfer functions)

With the prototype numbers,  $z = 0.4(0.5) + 0.3(-1) + (-0.2)(2) + 0.1 = -0.40$ . Then

| $\phi$        | step   | sigmoid | tanh    | ReLU   |
|---------------|--------|---------|---------|--------|
| $a = \phi(z)$ | 0.0000 | 0.4013  | -0.3799 | 0.0000 |

All four agree with `torch.sigmoid`, `torch.tanh` and `torch.relu` (checked in `code/dl/c01_perceptron.py`). OA version: inputs  $(1, 0, 1, 1)$ , weights  $(0.6, -0.9, -0.4, 0.5)$ , bias  $-0.3$ , sigmoid transfer. Then  $z = 0.40$  and the output is  $\sigma(z) = 0.5987$ . The input that is 0 contributes nothing whatever its weight,

which is exactly what the setter hopes you will forget.

**Remark 1.1.3 — Trap.** The neuron applies  $\phi$  to the weighted sum; it does not take a weighted sum of  $\phi(x_i)$ . A second common slip is dropping the bias: then every decision boundary is forced through the origin.

### §1.1.ii The biological analogy, and where it breaks

The name comes from a cartoon of a cortical neuron: dendrites collect signals from other cells, synapses scale them (the weights), the cell body integrates them (the sum), and the axon fires if the total crosses a threshold (the step function). The cartoon is useful for exactly one sentence. It breaks almost everywhere an interviewer might push:

- Real neurons communicate with spikes in time; information sits in rates and timing, not in one real number per forward pass.
- Dendrites themselves compute non-linear functions, so one biological neuron is closer to a small network than to one unit.
- Nobody has shown that the brain runs backpropagation: it would need the backward pass to reuse the forward weights exactly (“weight transport”).
- An artificial neuron with a sigmoid is literally logistic regression (see the logistic-regression chapter of the companion book *Classical Machine Learning*); the interesting behaviour comes from composing many of them and training them jointly.

**A neuron is a linear function followed by a fixed scalar non-linearity. Everything else is composition.**

## §1.2 The perceptron

*Prototypical example for this section:* Six labelled points in the plane; Rosenblatt’s rule adds misclassified points to  $\mathbf{w}$  until every point is on the right side.

### §1.2.i The algorithm

Rosenblatt’s perceptron [R58] is a step-function neuron  $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$  with labels  $y \in \{-1, +1\}$  and a learning rule that only reacts to mistakes.

#### Algorithm: Perceptron

Initialise  $\mathbf{w} = \mathbf{0}$ ,  $b = 0$ . Repeat over the data until an epoch makes no mistake: if  $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$  then  $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$  and  $b \leftarrow b + \eta y_i$ .

Why it is the right direction: after the update the new margin of the same point is  $y_i(\mathbf{w} + \eta y_i \mathbf{x}_i)^\top \mathbf{x}_i + y_i(b + \eta y_i) = y_i(\mathbf{w}^\top \mathbf{x}_i + b) + \eta(\|\mathbf{x}_i\|^2 + 1)$ , strictly larger than before. The rule is stochastic (sub)gradient descent on the **perceptron loss**  $\ell(\mathbf{w}, b) = \max(0, -y(\mathbf{w}^\top \mathbf{x} + b))$ , whose gradient is  $-y\mathbf{x}$  on a mistake and 0 otherwise.

code/dl/c01\_perceptron.py

```

1 def perceptron(X, y, epochs=20, lr=1.0):
2     w, b = np.zeros(X.shape[1]), 0.0
3     log = []
4     for ep in range(epochs):
5         mistakes = 0
6         for i in range(len(X)):
7             if y[i] * (w @ X[i] + b) <= 0:      # mistake (or on the boundary)
8                 w, b = w + lr * y[i] * X[i], b + lr * y[i]
9                 mistakes += 1
10                log.append((ep + 1, i + 1, w.copy(), b))
11        if mistakes == 0:
12            return w, b, log, ep + 1
13    return w, b, log, epochs

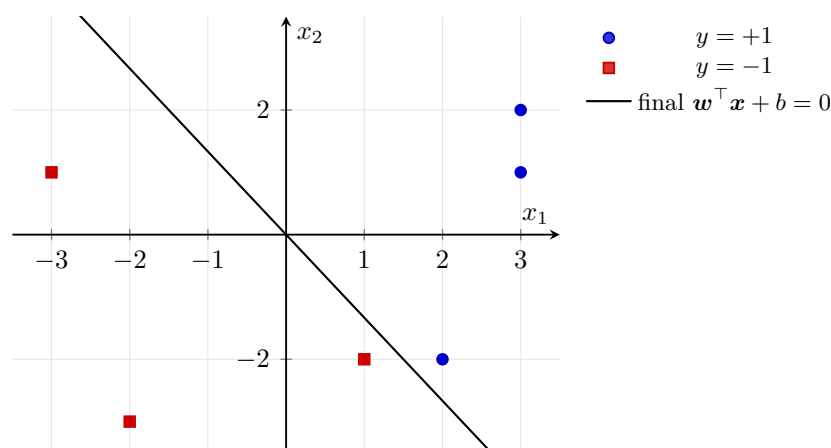
```

**Example 1.2.1** (A full trace)

Data:  $\mathbf{x}_1 = (3, 1)^+$ ,  $\mathbf{x}_2 = (1, -2)^-$ ,  $\mathbf{x}_3 = (3, 2)^+$ ,  $\mathbf{x}_4 = (-2, -3)^-$ ,  $\mathbf{x}_5 = (2, -2)^+$ ,  $\mathbf{x}_6 = (-3, 1)^-$ ;  $\eta = 1$ , points visited in index order. Every update made:

| epoch | mistake on     | $\mathbf{w}$ after | $b$ after |
|-------|----------------|--------------------|-----------|
| 1     | $\mathbf{x}_1$ | (3, 1)             | 1         |
| 1     | $\mathbf{x}_2$ | (2, 3)             | 0         |
| 1     | $\mathbf{x}_5$ | (4, 1)             | 1         |
| 2     | $\mathbf{x}_2$ | (3, 3)             | 0         |
| 2     | $\mathbf{x}_5$ | (5, 1)             | 1         |
| 3     | $\mathbf{x}_2$ | (4, 3)             | 0         |

Epoch 4 makes no mistake, so the algorithm stops after 6 updates at  $\mathbf{w} = (4, 3)$ ,  $b = 0$ . The result is identical to `sklearn.linear_model.Perceptron(eta0=1, shuffle=False)`, which the script asserts. Notice  $\mathbf{x}_2$  and  $\mathbf{x}_5$ , the two points closest to each other with opposite labels, cause every update after the first.



**Remark 1.2.2 — Trap.** “Increase the learning rate to make the perceptron converge faster.” With  $\mathbf{w}_0 = \mathbf{0}$ , every weight vector the algorithm visits is  $\eta$  times the one it visits with  $\eta = 1$ , so the signs of all predictions, the mistakes and the number of updates are identical. The script reruns the trace with  $\eta = 0.1$  and asserts the final

weights are exactly 0.1 times the ones above. The learning rate of the perceptron only matters when  $\mathbf{w}_0 \neq \mathbf{0}$ .

## §1.2.ii The convergence theorem

### Theorem 1.2.3 (Perceptron convergence)

Suppose  $\|(\mathbf{x}_i, 1)\| \leq R$  for all  $i$  and some unit vector  $\mathbf{w}^*$  (acting on augmented inputs  $(\mathbf{x}, 1)$ ) satisfies  $y_i \mathbf{w}^{*\top}(\mathbf{x}_i, 1) \geq \gamma > 0$  for all  $i$ . Then the perceptron makes at most  $(R/\gamma)^2$  updates, whatever the order of the data.

*Proof.* Absorb  $b$  into  $\mathbf{w}$  by appending 1 to every input, and take  $\eta = 1$  (by the remark above this costs nothing). Let  $\mathbf{w}_k$  be the weights after  $k$  updates, the  $k$ -th made on example  $i$ . Two inequalities:

- Progress:  $\mathbf{w}_k^\top \mathbf{w}^* = \mathbf{w}_{k-1}^\top \mathbf{w}^* + y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \mathbf{w}_{k-1}^\top \mathbf{w}^* + \gamma$ , so  $\mathbf{w}_k^\top \mathbf{w}^* \geq k\gamma$ .
- Bounded growth:  $\|\mathbf{w}_k\|^2 = \|\mathbf{w}_{k-1}\|^2 + 2y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2 \leq \|\mathbf{w}_{k-1}\|^2 + R^2$  because the update happened only when  $y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i \leq 0$ . So  $\|\mathbf{w}_k\|^2 \leq kR^2$ .

By Cauchy–Schwarz  $k\gamma \leq \mathbf{w}_k^\top \mathbf{w}^* \leq \|\mathbf{w}_k\| \leq \sqrt{k}R$ , hence  $k \leq (R/\gamma)^2$ .  $\square$

### Example 1.2.4 (How loose is the bound?)

For the six points above,  $R = 3.742$  and the best possible margin (from a hard-margin linear SVM, normalised) is  $\gamma = 0.277$ , so the bound is  $(R/\gamma)^2 \approx 182.0$  updates. The perceptron needed 6. The bound is a worst case over orderings and is driven by the single tightest pair of points.

**Question 1.2.5.** Why does the rule update when  $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$  rather than  $< 0$ ? (Start from  $\mathbf{w} = \mathbf{0}$ .)

**Remark 1.2.6 — Trap.** The theorem says nothing about non-separable data, where the perceptron never stops, and nothing about *which* separator it returns: unlike an SVM it does not maximise the margin, and the answer depends on the order of the data.

## §1.3 XOR: what one neuron cannot do

*Prototypical example for this section:*  $\text{XOR}(x_1, x_2)$  is 1 exactly when one input is 1; no line separates  $\{(0, 1), (1, 0)\}$  from  $\{(0, 0), (1, 1)\}$ .

### Proposition 1.3.1

No single step-function neuron computes XOR.

*Proof.* Suppose  $\mathbb{K}[w_1x_1 + w_2x_2 + b \geq 0]$  equals XOR. The four inputs give  $b < 0$ ,  $w_1 + w_2 + b < 0$ ,  $w_2 + b \geq 0$  and  $w_1 + b \geq 0$ . Adding the last two,  $w_1 + w_2 + 2b \geq 0$ , so  $w_1 + w_2 + b \geq -b > 0$ , contradicting the second.  $\square$

The same argument works for any monotone  $\phi$  thresholded at any level, so “use a sigmoid instead” does not help: a single neuron’s decision boundary is always a hyperplane. This observation, made precise by Minsky and Papert [MiPa69], is the usual one-line answer to “why do we need hidden layers?”.

### Example 1.3.2 (The perceptron on XOR)

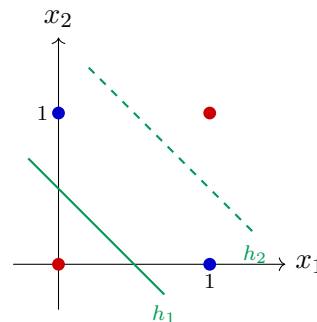
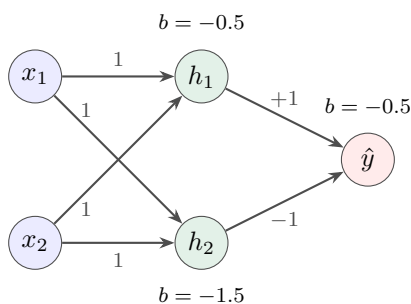
Running the perceptron for 50 epochs on XOR (labels  $\pm 1$ ) gives 200 updates in total and never fewer than 4 mistakes in an epoch: the weights cycle forever. `sklearn`’s perceptron ends at training accuracy 0.50, no better than a coin.

## §1.3.i Two layers solve it by hand

XOR = OR and not AND. Each of OR and AND is linearly separable, and so is “ $h_1$  and not  $h_2$ ” once  $h_1, h_2$  are available as features:

code/dl/c01\_perceptron.py

```
1 step = lambda t: (t >= 0).astype(float)
2 def xor_net(X):
3     h1 = step(X @ np.array([1, 1]) - 0.5)    # OR
4     h2 = step(X @ np.array([1, 1]) - 1.5)    # AND
5     return step(h1 - h2 - 0.5), h1, h2      # OR and not AND
```



| $x_1$ | $x_2$ | $h_1 = \text{OR}$ | $h_2 = \text{AND}$ | $\hat{y}$ |
|-------|-------|-------------------|--------------------|-----------|
| 0     | 0     | 0                 | 0                  | 0         |
| 0     | 1     | 1                 | 0                  | 1         |
| 1     | 0     | 1                 | 0                  | 1         |
| 1     | 1     | 1                 | 1                  | 0         |

The hidden layer *re-represents* the input: in  $(h_1, h_2)$  coordinates the four points become  $(0,0)$ ,  $(1,0)$ ,  $(1,0)$ ,  $(1,1)$  and the two classes are linearly separable. That is the whole idea of representation learning in one table.

### Example 1.3.3 (Can gradient descent find it?)

Train a 2- $H$ -1 network with tanh hidden units and a sigmoid output on the four XOR points with Adam, from 20 random initialisations. With  $H = 2$  (the minimum that can represent XOR) 10 of the 20 runs reach 100% accuracy; the rest stall in a poor local configuration. With  $H = 8$ , 20 of 20 succeed. Over-parameterising makes the optimisation problem easier, a theme that returns with every large model

in this book.

## §1.4 The multilayer perceptron

*Prototypical example for this section:* A 784-256-128-10 ReLU network for MNIST digits has 235,146 parameters.

### §1.4.i Forward pass and shapes

A **multilayer perceptron** (MLP, also **fully connected** or **dense** network) with layer widths  $n_0, n_1, \dots, n_L$  computes

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)}) \quad (l = 1, \dots, L),$$

with  $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$  and  $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ . The last layer usually has no  $\phi$  (it outputs logits; [chapter 2](#)). With a batch  $\mathbf{X} \in \mathbb{R}^{B \times n_0}$  stored row-wise, as every framework does, the same layer reads  $\mathbf{Z} = \mathbf{XW}^\top + \mathbf{1b}^\top$ , shape  $(B, n_l)$ ; PyTorch's `nn.Linear(n_in, n_out)` stores `weight` with shape  $(n_out, n_in)$  for exactly this reason.

#### Proposition 1.4.1 (Parameter count)

An MLP with widths  $n_0, \dots, n_L$  has  $\sum_{l=1}^L (n_{l-1} + 1) n_l$  parameters: each of the  $n_l$  units in layer  $l$  has  $n_{l-1}$  weights and one bias.

#### Example 1.4.2 (Counting an MNIST MLP)

For  $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ :  $785 \cdot 256 = 200,960$ ,  $257 \cdot 128 = 32,896$ ,  $129 \cdot 10 = 1,290$ ; total 235,146, equal to `sum(p.numel() for p in model.parameters())` for the PyTorch model. A 3-4-2 network has 26 parameters, a typical one-line OA item. Activation functions and the input layer have no parameters.

code/dl/c01\_perceptron.py

```
1 def mlp_params(sizes):
2     return sum(m * n + n for m, n in zip(sizes[:-1], sizes[1:]))
3 sizes = [784, 256, 128, 10]
4 n_ours = mlp_params(sizes)
```

### §1.4.ii Why the non-linearity is not optional

Delete every  $\phi$  and the network collapses:  $\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} = (\mathbf{W}^{(2)}\mathbf{W}^{(1)})\mathbf{x} + (\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})$ , a single affine map, however many layers you stack (the script checks this numerically). A deep linear network can represent only what one linear layer can, so it cannot solve XOR either.

**Remark 1.4.3 — Trap.** “ReLU is linear, so it adds no non-linearity.” ReLU is *piecewise* linear: it is linear on each side of 0 but not linear overall ( $\text{ReLU}(-1) + \text{ReLU}(1) \neq \text{ReLU}(0)$ ). A ReLU network is a piecewise-linear function with possibly a huge number of pieces, which is exactly what the next section exploits.

## §1.5 Universal approximation, and why depth still matters

*Prototypical example for this section:* Thirty-two ReLU units in one hidden layer approximate  $\sin(2\pi t) + t/2$  on  $[0, 1]$  to within 0.0048.

### Theorem 1.5.1 (Universal approximation, informal)

Let  $\phi$  be continuous and not a polynomial. For every continuous  $f : K \rightarrow \mathbb{R}$  on a compact  $K \subset \mathbb{R}^d$  and every  $\varepsilon > 0$  there is a network with one hidden layer of  $\phi$  units and a linear output whose output is within  $\varepsilon$  of  $f$  everywhere on  $K$ .

Cybenko proved it for sigmoids [Cy89]; later work extended it to any continuous non-polynomial activation. Two caveats make up the interview answer: the theorem says a suitable width *exists* but not how large it is (it can be exponential in  $d$ ), and it says nothing about whether gradient descent will find the weights or whether they generalise.

### §1.5.i A constructive proof in one dimension

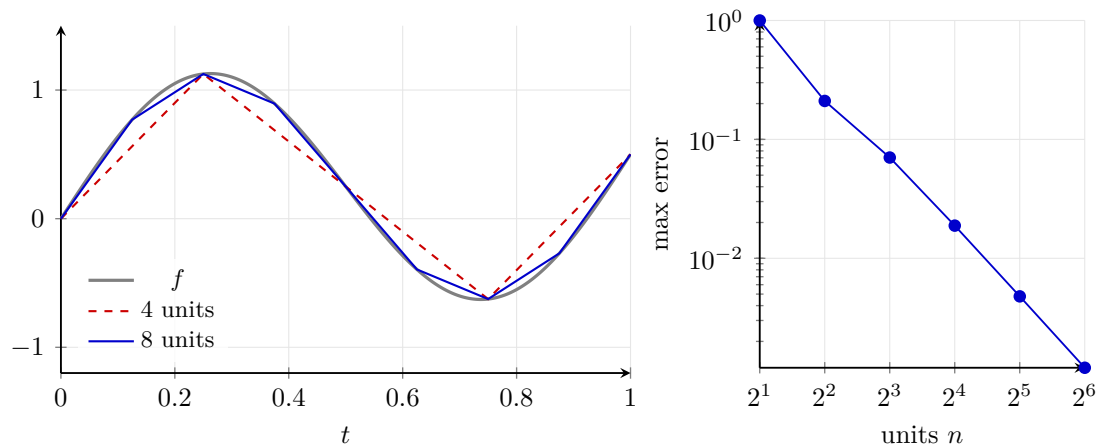
With ReLUs the 1-D case is a construction you can write in an interview. Take knots  $0 = t_0 < t_1 < \dots < t_n = 1$  and the piecewise-linear interpolant of  $f$ . Its slope on  $[t_k, t_{k+1}]$  is  $s_k$ . Then

$$\hat{f}(t) = f(0) + \sum_{k=0}^{n-1} a_k \operatorname{ReLU}(t - t_k), \quad a_0 = s_0, \quad a_k = s_k - s_{k-1},$$

because the unit  $\operatorname{ReLU}(t - t_k)$  switches on at  $t_k$  and changes the slope by  $a_k$  there. That is a one-hidden-layer network with  $n$  units: hidden weights 1, hidden biases  $-t_k$ , output weights  $a_k$ .

code/dl/c01\_perceptron.py

```
1 def relu_net_interp(f, n):
2     """One hidden ReLU layer with n units that interpolates f at n+1 knots."""
3     knots = np.linspace(0, 1, n + 1)
4     vals = f(knots)
5     slopes = np.diff(vals) / np.diff(knots)
6     a = np.r_[slopes[0], np.diff(slopes)] # output weights
7     def net(t):
8         H = np.maximum(0.0, t[:, None] - knots[:-1][None, :]) # hidden layer
9         return vals[0] + H @ a
10    return net
```



### Example 1.5.2 (Error rate of the construction)

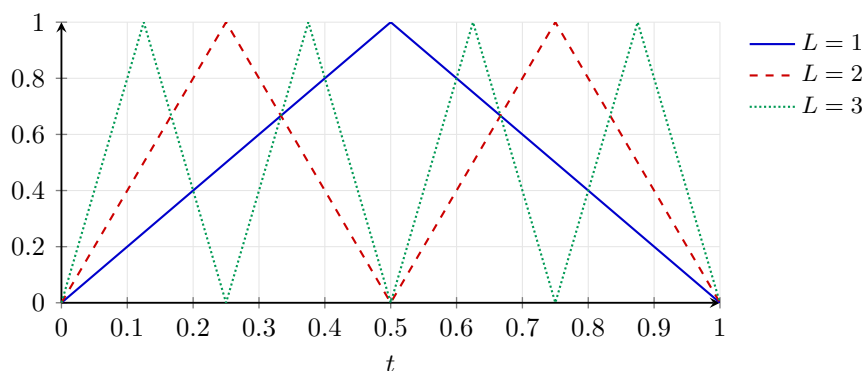
The script verifies that the network equals `numpy.interp` at 401 points and measures the worst error: 0.2105, 0.0704, 0.0188, 0.0048, 0.0012 for  $n = 4, 8, 16, 32, 64$ . Doubling  $n$  divides the error by about 3.98, matching the interpolation bound  $h^2 \max |f''|/8$  with  $h = 1/n$ ; for  $n = 64$  that bound is 0.0012. In  $d$  dimensions the same grid argument needs  $n^d$  cells: the curse of dimensionality hiding inside “universal”.

## §1.5.ii Depth buys pieces exponentially

A one-hidden-layer ReLU net on  $\mathbb{R}$  with  $m$  units is piecewise linear with at most  $m + 1$  pieces (each unit adds at most one kink). Now compose the tent map  $g(t) = 2 \operatorname{ReLU}(t) - 4 \operatorname{ReLU}(t - \frac{1}{2})$ , which uses 2 ReLU units and maps  $[0, 1]$  onto  $[0, 1]$  up and back down. Composing it  $L$  times gives a sawtooth.

code/dl/c01\_perceptron.py

```
1 def tent_layer(t):          # 2 ReLU units: g(t) = 2 relu(t) - 4 relu(t - 1/2)
2     return 2 * np.maximum(0, t) - 4 * np.maximum(0, t - 0.5)
3 def deep_tent(t, L):
4     for _ in range(L):
5         t = tent_layer(t)
6     return t
```



Counting slope changes numerically, depth  $L = 1, \dots, 6$  gives 2, 4, 8, 16, 32, 64 linear pieces, i.e.  $2^L$ , using only  $2L$  units. The depth-6 sawtooth uses 12 units; a single hidden layer needs at least 63 to produce 64 pieces. Depth multiplies, width adds.

**One hidden layer is enough in principle; depth is what makes it enough in practice.**

## §1.6 Capacity: what makes a network more expressive

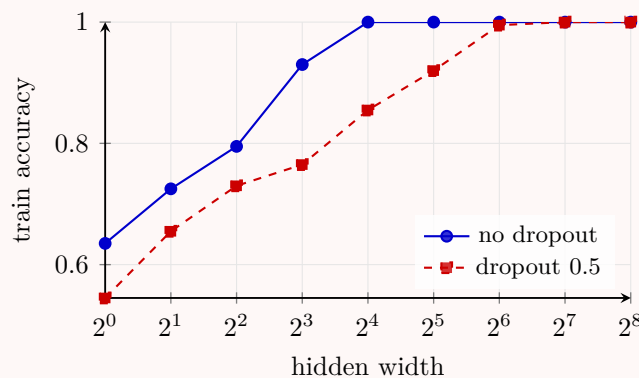
*Prototypical example for this section:* “Which of these increases a network’s capacity: more layers, dropout, a larger learning rate, more data?” Only the first.

**Capacity** is the richness of the set of functions the architecture can represent (its hypothesis class). It is a property of the model, not of the data or of the optimiser.

- **Increase capacity:** more layers, more units per layer, more input features, a richer activation, fewer constraints (e.g. removing weight sharing).
- **Reduce effective capacity:** dropout, weight decay, early stopping, data augmentation, weight sharing, bottleneck layers. These restrict which functions training can reach or prefers.
- **Do not change capacity:** the learning rate, the batch size, the number of epochs as such, and the amount of data. They change what training *finds*, not what the model *can* represent.

### Example 1.6.1 (Memorising random labels)

Give 200 random points in  $\mathbb{R}^{10}$  coin-flip labels, so the only way to fit them is to memorise. Train one-hidden-layer ReLU nets of increasing width to convergence. Training accuracy rises from 63.5% at width 1 to 100.0% at width 16. With dropout  $p = 0.5$  the width-16 net reaches only 85.5%: the same architecture, lower effective capacity.



**Remark 1.6.2 — Trap.** “Dropout increases capacity because it trains an ensemble.” Dropout trains an implicit ensemble of thinned sub-networks that share weights; each sub-network is smaller than the full one, and the averaged predictor is pulled toward simpler functions. It is a regulariser: it lowers effective capacity (chapter 5).

**Remark 1.6.3 — Trap.** “A bigger learning rate lets the model fit more complex functions.” It changes the path of optimisation, sometimes for the better (noise that escapes sharp minima), but the set of representable functions is fixed by the

architecture.

## §1.7 A few problems to think about

**Problem 1A.** How many trainable parameters does a fully connected network with layer sizes  $100 \rightarrow 50 \rightarrow 50 \rightarrow 3$  have (biases included)?

[hint · solution](#)

**Problem 1B.** Give weights and a bias for a single step neuron computing NAND on  $\{0, 1\}^2$ . Why does this show that networks of step neurons can compute any Boolean function?

[hint · solution](#)

**Problem 1C.** A perceptron *without bias* is trained from  $\mathbf{w} = \mathbf{0}$  on linearly separable data. Every input is then multiplied by  $c > 0$  and training is rerun. How do  $R$ ,  $\gamma$ , the bound  $(R/\gamma)^2$  and the sequence of mistakes change?

[hint · solution](#)

**Problem 1D.** Show that the one-hidden-layer ReLU network on  $\mathbb{R}$

$$f(t) = c + \sum_{k=1}^m a_k \text{ReLU}(w_k t + b_k)$$

is piecewise linear with at most  $m + 1$  pieces.

[hint · solution](#)

**Problem 1E.** Which statements are true? (a) A single sigmoid neuron can represent XOR. (b) Stacking linear layers without activations increases representational power. (c) Adding hidden layers increases capacity. (d) Dropout increases capacity. (e) A one-hidden-layer network can approximate any continuous function on a compact set, given enough units.

[hint · solution](#)

**Problem 1F.** Prove that the perceptron update never decreases  $\mathbf{w}^\top \mathbf{w}^*$  and explain in one sentence why that alone does not prove convergence.

[hint · solution](#)

# 2 Activations, losses and output layers

Three choices sit at fixed places in every network: the non-linearity between layers, the function that turns the last layer’s numbers into a prediction, and the loss that scores that prediction. OAs test all three with one-minute numerics (“compute the softmax of these five logits”) and with classification questions (“which of these is a loss and which is an activation?”). Interviews test whether you know why the standard pairings are standard.

## §2.1 Activation functions

*Prototypical example for this section:* Sigmoid, tanh, ReLU and GELU evaluated at  $z = -2$  give four different answers, and the derivative of the sigmoid is never larger than 0.25.

### §2.1.i What the activation is for

Chapter 1 showed that without an activation a deep network collapses to one affine map. The activation is the *only* source of non-linearity in an MLP: weights and biases enter linearly, and the loss is applied after the network, so it cannot change what functions the network represents.

**Remark 2.1.1 — Trap.** “Which component provides non-linearity: weights, biases, activation function, loss function?” Only the activation function. A non-linear *loss* (cross-entropy is very non-linear) does not help: a linear model trained with cross-entropy is still logistic regression, with a linear boundary.

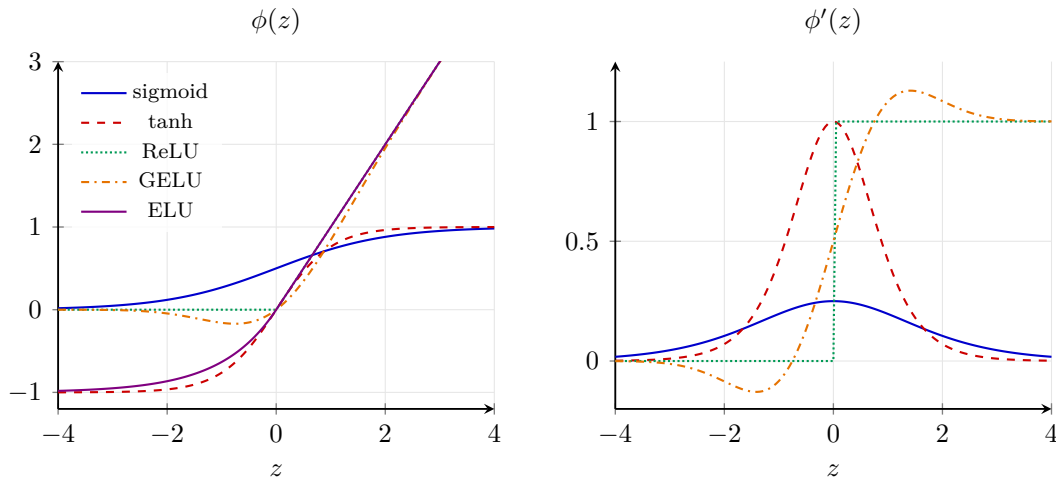
### §2.1.ii The zoo, with values

| name         | $\phi(z)$                                         | $\phi'(z)$                                  |
|--------------|---------------------------------------------------|---------------------------------------------|
| sigmoid      | $\sigma(z) = 1/(1 + e^{-z})$                      | $\sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$ |
| tanh         | $(e^z - e^{-z})/(e^z + e^{-z}) = 2\sigma(2z) - 1$ | $1 - \tanh^2 z \leq 1$                      |
| ReLU         | $\max(0, z)$                                      | $\mathbb{K}[z > 0]$                         |
| leaky ReLU   | $\max(\alpha z, z), \alpha \approx 0.01$          | 1 or $\alpha$                               |
| ELU          | $z$ if $z > 0$ , else $\alpha(e^z - 1)$           | 1 or $\alpha e^z$                           |
| GELU         | $z \Phi(z)$ , $\Phi$ the standard normal CDF      | $\Phi(z) + z \varphi(z)$                    |
| SiLU / swish | $z \sigma(z)$                                     | $\sigma(z)(1 + z(1 - \sigma(z)))$           |
| softplus     | $\log(1 + e^z)$                                   | $\sigma(z)$                                 |

#### Example 2.1.2 (Values an OA might ask for)

Computed by `code/dl/c02_activations.py`, each function checked against `torch.nn.functional` on a grid of 161 points and each derivative against autograd:

| $\phi$               | $z = -2$ | $-0.5$  | 0      | 0.5    | 2      |
|----------------------|----------|---------|--------|--------|--------|
| sigmoid              | 0.1192   | 0.3775  | 0.5000 | 0.6225 | 0.8808 |
| tanh                 | -0.9640  | -0.4621 | 0.0000 | 0.4621 | 0.9640 |
| ReLU                 | 0.0000   | 0.0000  | 0.0000 | 0.5000 | 2.0000 |
| leaky ReLU (0.01)    | -0.0200  | -0.0050 | 0.0000 | 0.5000 | 2.0000 |
| ELU ( $\alpha = 1$ ) | -0.8647  | -0.3935 | 0.0000 | 0.5000 | 2.0000 |
| GELU                 | -0.0455  | -0.1543 | 0.0000 | 0.3457 | 1.9545 |
| SiLU                 | -0.2384  | -0.1888 | 0.0000 | 0.3112 | 1.7616 |



### §2.1.iii Sigmoid and tanh: saturation

Both squash  $\mathbb{R}$  into a bounded interval, and both are nearly flat away from 0:  $\sigma'(5) = 0.0066$ . Backpropagation multiplies one such factor per layer (chapter 3), and since  $\sigma' \leq \frac{1}{4}$  ten sigmoid layers shrink a gradient by at least  $4^{10}$ , i.e. to at most  $9.54 \times 10^{-7}$  of its size from that factor alone. This is the **vanishing gradient** problem in its simplest form.

The sigmoid has a second defect: its outputs are all positive. For a neuron fed by sigmoid units,  $\partial \mathcal{L} / \partial w_i = \delta \cdot h_i$  with every  $h_i > 0$ , so all of that neuron's weight gradients share the sign of  $\delta$  and the weights can only move together up or together down, giving zig-zag updates. tanh is zero-centred and its maximum slope is 1, so it was preferred for hidden layers even before ReLU. Today sigmoid is used at outputs (probabilities) and in gates (LSTM, chapter 7), almost never in hidden layers.

### §2.1.iv ReLU and its relatives

ReLU does not saturate for  $z > 0$  (derivative exactly 1), is cheap, and produces exact zeros (sparse activations). Its failure mode is the **dead ReLU**: a unit whose pre-activation is negative on every training input gets zero gradient forever (chapter 3 has a simulation). Leaky ReLU and PReLU (learned  $\alpha$ ) keep a small slope for  $z < 0$ ; ELU makes the negative side smooth and pushes mean activations toward zero.

**Remark 2.1.3 — Trap.** “ReLU is not differentiable at 0, so it cannot be trained by gradient descent.” The kink is a single point that floating-point inputs essentially never hit exactly; frameworks use the subgradient 0 there, and ReLU is the default

hidden activation of most CNNs.

### §2.1.v Smooth modern activations: GELU and SiLU

GELU weights the input by the probability that a standard normal falls below it,  $z \Phi(z)$ ; Transformers (BERT, GPT-style models) use it or SiLU in their feed-forward blocks [HeGi16]. The common approximation  $\frac{1}{2}z(1 + \tanh(\sqrt{2/\pi}(z + 0.044715z^3)))$  is within 0.00047 of the exact GELU on  $[-6, 6]$ . Both are *non-monotone*: SiLU has a minimum  $-0.278$  at  $z = -1.278$ , GELU a minimum  $-0.170$  at  $z = -0.752$  (found numerically). They behave like ReLU for large  $|z|$  but pass small negative signals and have a continuous derivative.

**Hidden layers: ReLU for CNNs and MLPs, GELU/SiLU for Transformers, tanh and sigmoid only inside recurrent cells and gates. Output layers: chosen by the task, next section.**

## §2.2 Output layers: sigmoid and softmax

*Prototypical example for this section:* Logits  $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$  for five classes; the softmax gives class 1 probability 0.5585.

### §2.2.i Softmax

For  $K$  classes the network outputs **logits**  $\mathbf{z} \in \mathbb{R}^K$  and

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

It is positive and sums to 1, preserves the order of the logits, and is invariant to adding a constant to every logit (multiply top and bottom by  $e^{-c}$ ), but not to scaling them.

#### Example 2.2.1 (Softmax confidence from five logits)

For  $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$ :  $e^{\mathbf{z}} = (7.3891, 2.7183, 1.1052, 0.3679, 1.6487)$  with sum 13.2291, so

$$\text{softmax}(\mathbf{z}) = (0.5585, 0.2055, 0.0835, 0.0278, 0.1246).$$

The model predicts class 1 with confidence 55.9%. Note that a logit gap of 1 (2 vs 1) is a probability ratio of  $e \approx 2.72$ , not a difference of 1 in anything.

### §2.2.ii Overflow and the log-sum-exp trick

$e^{1000}$  overflows double precision, so the textbook formula applied to  $\mathbf{z} = (1000, 1001, 1002)$  returns *NaN* for every entry. The fix uses shift invariance: subtract  $m = \max_k z_k$  first. Then every exponent is  $\leq 0$ , the largest term is exactly 1, and nothing overflows. The same trick gives a stable **log-sum-exp**

$$\text{LSE}(\mathbf{z}) = \log \sum_k e^{z_k} = m + \log \sum_k e^{z_k - m},$$

which is what a cross-entropy implementation actually needs.

code/dl/c02\_activations.py

```

1 def softmax(z):
2     z = z - z.max(axis=-1, keepdims=True)          # shift: same result, no overflow
3     e = np.exp(z)
4     return e / e.sum(axis=-1, keepdims=True)
5
6 def logsumexp(z):
7     m = z.max(axis=-1, keepdims=True)
8     return (m + np.log(np.exp(z - m).sum(axis=-1, keepdims=True))).squeeze(-1)
9
10 def cross_entropy(z, y):
11     # z: (B, K) logits, y: (B,) ints
    return np.mean(logsumexp(z) - z[np.arange(len(y)), y])

```

For (1000, 1001, 1002) the stable version returns (0.090, 0.245, 0.665), identical to  $\text{softmax}(0, 1, 2)$ , and  $\text{LSE} = 1002.4076$ ; both match PyTorch’s `softmax` and SciPy’s `logsumexp`.

### §2.2.iii Temperature

Dividing logits by a **temperature**  $T > 0$  before the softmax sharpens ( $T < 1$ ) or flattens ( $T > 1$ ) the distribution without changing the arg-max. For the five logits above (last column: entropy in nats; the uniform distribution has  $\ln 5 = 1.609$ ):

| $T$ | $p_1$ | $p_2$ | $p_3$ | $p_4$ | $p_5$ | $H(p)$ |
|-----|-------|-------|-------|-------|-------|--------|
| 0.5 | 0.826 | 0.112 | 0.018 | 0.002 | 0.041 | 0.620  |
| 1   | 0.559 | 0.205 | 0.084 | 0.028 | 0.125 | 1.217  |
| 2   | 0.372 | 0.226 | 0.144 | 0.083 | 0.176 | 1.495  |
| 5   | 0.264 | 0.216 | 0.180 | 0.145 | 0.195 | 1.590  |

As  $T \rightarrow 0$  the softmax becomes a one-hot arg-max, as  $T \rightarrow \infty$  it becomes uniform. Temperature reappears in knowledge distillation and LLM sampling (chapter 9).

### §2.2.iv Sigmoid is two-class softmax

$\text{softmax}(a, b)_1 = e^a / (e^a + e^b) = 1 / (1 + e^{-(a-b)}) = \sigma(a - b)$  (checked numerically). So a binary classifier needs one logit and a sigmoid; two logits and a softmax is equivalent but over-parameterised.

### §2.2.v The softmax Jacobian

With  $\mathbf{p} = \text{softmax}(\mathbf{z})$ ,  $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$ , i.e.  $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$  (verified against `torch.autograd.functional.jacobian`). Rows sum to zero because the outputs always sum to 1. In practice you never form  $J$ : combined with cross-entropy it collapses to the gradient  $\mathbf{p} - \mathbf{y}$  derived below.

**Remark 2.2.2 — Trap.** Multi-label is not multi-class. If an image can contain both a cat and a dog, use  $K$  independent sigmoids with  $K$  binary cross-entropies. A softmax forces the probabilities to compete and sum to 1, so “cat 0.9, dog 0.8” is impossible under it.

## §2.3 Loss functions

*Prototypical example for this section:* True class 1 for the logits above: cross-entropy  $-\ln 0.5585 = 0.5824$  nats.

### §2.3.i Cross-entropy is maximum likelihood

If the network outputs a distribution  $p_\theta(\cdot | \mathbf{x})$  over  $K$  classes and the data are independent, the negative log-likelihood of the labels is

$$-\sum_i \log p_\theta(y_i | \mathbf{x}_i) = \sum_i (\text{LSE}(\mathbf{z}_i) - z_{i,y_i}),$$

the **categorical cross-entropy** between the one-hot label and  $\mathbf{p}$ . Minimising it is minimising the KL divergence from the empirical label distribution to the model (the information-theory section of the companion book *Classical Machine Learning* has the identity  $H(q, p) = H(q) + \text{KL}(q||p)$ ).

#### Proposition 2.3.1 (Gradient of softmax + cross-entropy)

For one example with logits  $\mathbf{z}$ , one-hot label  $\mathbf{y}$  and  $\mathcal{L} = -\sum_k y_k \log \text{softmax}(\mathbf{z})_k$ ,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \text{softmax}(\mathbf{z}) - \mathbf{y}.$$

*Proof.*  $\mathcal{L} = \text{LSE}(\mathbf{z}) - \mathbf{z}^\top \mathbf{y}$  because  $\sum_k y_k = 1$ . The gradient of LSE is the softmax, and the gradient of  $\mathbf{z}^\top \mathbf{y}$  is  $\mathbf{y}$ .  $\square$

For [example 2.2.1](#) with true class 1 the gradient, identical to autograd's, is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = (-0.4415, 0.2055, 0.0835, 0.0278, 0.1246).$$

The gradient pushes the true logit up by  $1 - p_y$  and each wrong logit down by its probability: big mistakes get big corrections, and the gradient never saturates while the prediction is wrong.

#### Example 2.3.2 (Cross-entropy from logits, many cases)

Each row is checked against `F.cross_entropy`. Classes are numbered from 0 here, as in code.

| logits $\mathbf{z}$    | $y$ | $\text{softmax}(\mathbf{z})$        | pred | $p_y$ | $\text{CE} = -\ln p_y$ |
|------------------------|-----|-------------------------------------|------|-------|------------------------|
| (3, 1, 0.2)            | 0   | (0.836, 0.113, 0.051)               | 0    | 0.836 | 0.179                  |
| (0, 0, 0, 0)           | 2   | (0.250, 0.250, 0.250, 0.250)        | 0    | 0.250 | 1.386                  |
| (2, 2, -1)             | 1   | (0.488, 0.488, 0.024)               | 0    | 0.488 | 0.718                  |
| (5, -2, 0, 1)          | 1   | (0.975, 0.001, 0.007, 0.018)        | 0    | 0.001 | 7.026                  |
| (1, 2, 3, 4, 5)        | 0   | (0.012, 0.032, 0.086, 0.234, 0.636) | 4    | 0.012 | 4.452                  |
| (-1, 0.5, 0.5, 2.5, 0) | 3   | (0.022, 0.098, 0.098, 0.723, 0.059) | 3    | 0.723 | 0.324                  |
| (10, 0)                | 0   | (1.000, 0.000)                      | 0    | 1.000 | 0.000                  |

Patterns worth memorising: equal logits give  $\text{CE} = \ln K$  (row 2); a confident wrong prediction is punished hard (row 4: the true class has probability 0.001); the loss of a correct prediction is small but never exactly 0 (last row); with the same probabilities the loss of predicting the wrong class grows by the logit gap (CE for

class 4 in [example 2.2.1](#) is  $3.5824 = 0.5824 + 3$ ).

**Remark 2.3.3 — Trap.** Sanity check at initialisation: with small random weights the logits are nearly equal, so the loss of a  $K$ -class classifier starts near  $\ln K$ . For a batch of 64 random 10-class logits of scale 0.01 the script measures 2.3014 against  $\ln 10 = 2.3026$ . A first loss far from  $\ln K$  means a bug (wrong labels, softmax applied twice, a huge initial scale).

### §2.3.ii Binary cross-entropy and its stable form

For  $y \in \{0, 1\}$  and  $p = \sigma(z)$ ,  $\text{BCE} = -[y \log p + (1 - y) \log(1 - p)]$ : 0.1054 for  $p = 0.9$ ,  $y = 1$ ; 2.3026 for  $p = 0.1$ ,  $y = 1$ ;  $\ln 2 = 0.6931$  at  $p = 0.5$ ; and 4.6052 for a 0.99 prediction of the wrong class. Computing  $\sigma$  first and then log breaks for large  $|z|$ : at  $z = -1000$ ,  $y = 1$  the sigmoid underflows to 0 and the loss becomes  $\infty$ . Working from the logit,

$$\text{BCE}(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}),$$

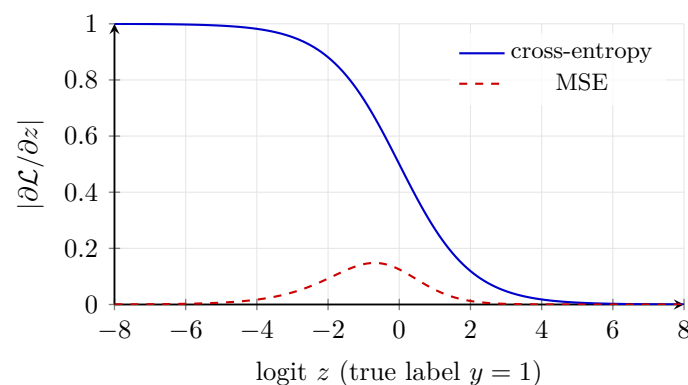
gives the correct 1000.0. This is `BCEWithLogitsLoss`, and the reason you pass logits, not probabilities, to it.

code/dl/c02\_activations.py

```
1 def bce_with_logits(z, y):          # = -[y log s(z) + (1-y) log(1-s(z))], stable
2     return np.maximum(z, 0) - z * y + np.log1p(np.exp(-np.abs(z)))
```

### §2.3.iii Why not MSE for classification?

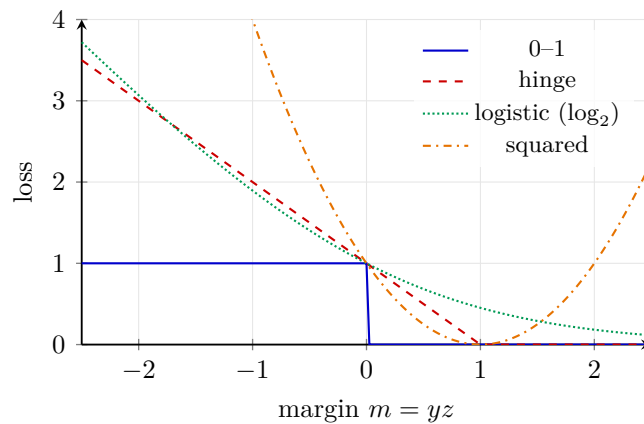
With a sigmoid output and squared error  $\frac{1}{2}(\sigma(z) - y)^2$ , the gradient is  $(\sigma(z) - y) \sigma'(z)$ . When the prediction is confidently wrong ( $y = 1$ ,  $z = -5$ ) the factor  $\sigma'(z)$  kills it: 0.0066 for MSE against 0.9933 for cross-entropy. The model that is most wrong learns slowest under MSE. Cross-entropy's log exactly cancels the sigmoid's saturation. (MSE on a sigmoid is also non-convex in  $z$ ; CE is convex.)



### §2.3.iv Margin losses: hinge

With labels  $y \in \{-1, +1\}$  and score  $z$ , every binary loss is a function of the **margin**  $m = yz$ . The 0-1 loss  $\mathbb{1}[m \leq 0]$  is what we care about but has zero gradient almost everywhere; the **hinge**  $\max(0, 1 - m)$  (SVMs) and the logistic loss  $\log(1 + e^{-m})$  are convex

upper bounds (the logistic in base 2 so that it passes through  $(0, 1)$ ). The multiclass hinge sums  $\max(0, z_j - z_y + 1)$  over wrong classes  $j$ ; for [example 2.2.1](#) with true class 2 it is 2.60 (`F.multi_margin_loss` divides by  $K$ ).



The squared loss punishes *correct* predictions with margin above 1: another reason it is a poor classification loss.

### §2.3.v Regression: MSE, MAE, Huber

MSE is the Gaussian negative log-likelihood and is dominated by outliers; MAE (Laplace likelihood) is robust but has a constant-size gradient that does not shrink near the optimum. [Huber](#) interpolates: quadratic within  $\delta$ , linear outside.

code/dl/c02\_activations.py

```
1 def huber(r, delta=1.0):
2     a = np.abs(r)
3     return np.where(a <= delta, 0.5 * r ** 2, delta * (a - 0.5 * delta))
```

With  $\delta = 1$ , a residual of 10 costs 9.5 under Huber and 50.0 under  $\frac{1}{2}r^2$ ; a residual of 0.2 costs 0.02 under both (matches `F.huber_loss`).

### §2.3.vi Focal loss and label smoothing

**Focal loss** [\[Li+17\]](#) multiplies cross-entropy by  $(1 - p_y)^\gamma$  to down-weight easy, well-classified examples, which dominate the loss when one class (background) vastly outnumbers the rest:

| $p_y$ | CE     | focal ( $\gamma = 2$ ) | factor $(1 - p_y)^2$ |
|-------|--------|------------------------|----------------------|
| 0.9   | 0.1054 | 0.0011                 | 0.01                 |
| 0.5   | 0.6931 | 0.1733                 | 0.25                 |
| 0.1   | 2.3026 | 1.8651                 | 0.81                 |

**Label smoothing** with parameter  $\varepsilon$  replaces the one-hot target by  $(1 - \varepsilon)\mathbf{y} + \varepsilon/K$ ; with  $\varepsilon = 0.1$ ,  $K = 5$  the target is 0.92 on the true class and 0.02 elsewhere. The model can no longer drive the loss to 0 by pushing one logit to infinity, which curbs over-confidence. For [example 2.2.1](#) the smoothed loss is 0.7304 (`F.cross_entropy(..., label_smoothing=0.1)` agrees).

### §2.3.vii Which is a loss, which is an activation?

| activations                                               | output maps                                | losses                                                                     |
|-----------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------|
| ReLU, leaky ReLU, ELU, GELU, SiLU, tanh, sigmoid (hidden) | sigmoid, softmax, identity (linear output) | MSE, MAE, Huber, BCE, CE, hinge, focal, KL divergence, contrastive/triplet |

Sigmoid and softmax appear in two columns because they are used both as squashing functions and as output maps; they are never losses.

| task                    | output layer             | loss              |
|-------------------------|--------------------------|-------------------|
| regression              | linear (identity)        | MSE / MAE / Huber |
| binary classification   | 1 logit, sigmoid         | BCE (with logits) |
| multi-class (one label) | $K$ logits, softmax      | categorical CE    |
| multi-label             | $K$ logits, $K$ sigmoids | sum of $K$ BCEs   |
| counts                  | exponential (log-rate)   | Poisson NLL       |

**Remark 2.3.4 — Trap.** In PyTorch, `nn.CrossEntropyLoss` takes *logits* and applies log-softmax internally. Putting an `nn.Softmax` before it computes softmax twice; training still runs but gradients shrink and the loss floor is wrong. The same holds for `BCEWithLogitsLoss` after a sigmoid (chapter 11 lists this among the silent bugs).

## §2.4 A few problems to think about

**Problem 2A.** A 4-class classifier outputs logits  $(1, 1, 1, 1)$ . What is the cross-entropy loss? What if the logits are  $(101, 101, 101, 101)$ ?

hint · solution

**Problem 2B.** Show  $\tanh z = 2\sigma(2z) - 1$  and deduce  $\tanh'(0) = 4\sigma'(0)$ .

hint · solution

**Problem 2C.** Derive  $\partial \text{BCE} / \partial z = \sigma(z) - y$  for  $\text{BCE} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$ , and verify that the stable form  $\max(z, 0) - zy + \log(1 + e^{-|z|})$  equals BCE for both signs of  $z$ .

hint · solution

**Problem 2D.** Which are loss functions? (a) ReLU (b) hinge (c) softmax (d) Huber (e) cross-entropy (f) GELU (g) KL divergence.

hint · solution

**Problem 2E.** A softmax classifier predicts  $p_y = 0.2$  for the true class. By how much must the true logit increase, other logits fixed, to halve the loss? Give a formula, then evaluate for  $p_y = 0.2$ .

hint · solution

**Problem 2F.** A team trains an image tagger where each image may have several of 20 tags, with a softmax output and categorical cross-entropy. What goes wrong, and what is the fix?

hint · solution

# 3 Backpropagation

## §3.1 Backpropagation

[STUB: Backpropagation]



# 4 Optimisers and learning-rate schedules

## §4.1 Optimisers and learning-rate schedules

[STUB: Optimisers and learning-rate schedules]



# 5 Initialisation, normalisation and regularisation

## §5.1 Initialisation, normalisation and regularisation

[STUB: Initialisation, normalisation and regularisation]



# II

## Architectures

## Part II: Contents

---

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>6</b> | <b>Convolutional networks</b>           | <b>35</b> |
| 6.1      | Convolutional networks . . . . .        | 35        |
| <b>7</b> | <b>Recurrent networks</b>               | <b>37</b> |
| 7.1      | Recurrent networks . . . . .            | 37        |
| <b>8</b> | <b>Attention and the Transformer</b>    | <b>39</b> |
| 8.1      | Attention and the Transformer . . . . . | 39        |

---

# 6 Convolutional networks

## §6.1 Convolutional networks

[STUB: Convolutional networks]



# 7 Recurrent networks

## §7.1 Recurrent networks

[STUB: Recurrent networks]



# 8 Attention and the Transformer

## §8.1 Attention and the Transformer

[STUB: Attention and the Transformer]



# III

## Modern Deep Learning

## Part III: Contents

---

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>9</b>  | <b>Language models and LLMs</b>                  | <b>43</b> |
| 9.1       | Language models and LLMs . . . . .               | 43        |
| <b>10</b> | <b>Generative and representation learning</b>    | <b>45</b> |
| 10.1      | Generative and representation learning . . . . . | 45        |

---

# 9 Language models and LLMs

## §9.1 Language models and LLMs

[STUB: Language models and LLMs]



# 10 Generative and representation learning

## §10.1 Generative and representation learning

[STUB: Generative and representation learning]



# IV

## Interview Mode

## Part IV: Contents

---

|           |                                |           |
|-----------|--------------------------------|-----------|
| <b>11</b> | <b>Practice and PyTorch</b>    | <b>49</b> |
| 11.1      | Practice and PyTorch . . . . . | 49        |
| <b>12</b> | <b>Implement it</b>            | <b>51</b> |
| 12.1      | Implement it . . . . .         | 51        |
| <b>13</b> | <b>Rapid-fire</b>              | <b>53</b> |
| 13.1      | Rapid-fire . . . . .           | 53        |
| <b>14</b> | <b>Problems</b>                | <b>55</b> |
| 14.1      | Problems . . . . .             | 55        |

---

# 11 Practice and PyTorch

## §11.1 Practice and PyTorch

[STUB: Practice and PyTorch]



# 12 Implement it

## §12.1 Implement it

[STUB: Implement it]



# 13 Rapid-fire

## §13.1 Rapid-fire

[STUB: Rapid-fire]



# 14 Problems

## §14.1 Problems

[STUB: Problems]





Backmatter



# A Hints to problems

- 1A.** Each layer contributes  $(\text{inputs} + 1) \times \text{outputs}$ .
- 1B.** NAND is 0 only at  $(1, 1)$ .
- 1C.** Both  $R$  and  $\gamma$  are lengths in input space.
- 1D.** Where can the slope of  $f$  change?
- 1E.** Two are true.
- 1F.** The other half of the proof bounds  $\|\mathbf{w}\|$ .
- 2A.** Shift invariance.
- 2B.** Write  $\tanh z = (1 - e^{-2z})/(1 + e^{-2z})$ .
- 2C.**  $\log \sigma(z) = -\log(1 + e^{-z})$  and  $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$ .
- 2D.** Four of them.
- 2E.** Let  $S$  be the sum of the other exponentials;  $p_y = e^{z_y}/(e^{z_y} + S)$ .
- 2F.** What does a softmax force the 20 probabilities to do?



# B Solutions to problems

- 1A.**  $101 \cdot 50 = 5050$ ,  $51 \cdot 50 = 2550$ ,  $51 \cdot 3 = 153$ ; total 7753.
- 1B.**  $\mathbf{w} = (-2, -2)$ ,  $b = 3$ : the sums are 3, 1, 1,  $-1$  on  $(0, 0)$ ,  $(0, 1)$ ,  $(1, 0)$ ,  $(1, 1)$ , so the output is 1, 1, 1, 0. NAND is functionally complete (every Boolean circuit can be rewritten with NAND gates only), and each gate is one neuron, so any Boolean function is a feed-forward network of step neurons; the depth is the circuit's depth.
- 1C.**  $R$  and  $\gamma$  both scale by  $c$ , so the bound is unchanged. The run itself is identical up to scale: by induction every weight vector is  $c$  times the original one, so  $\text{sign}(\mathbf{w}^\top c\mathbf{x})$  equals the original prediction and the same points are misclassified in the same order. (With a bias the constant coordinate 1 is not scaled and exact invariance is lost.)
- 1D.** Each term is linear except at  $t = -b_k/w_k$  (if  $w_k \neq 0$ ), where its slope jumps. So  $f$  is linear between consecutive elements of the set of at most  $m$  such breakpoints, giving at most  $m + 1$  intervals. Contrast with the  $2^L$  pieces of the depth- $L$  sawtooth built from  $2L$  units.
- 1E.** (c) and (e). (a) fails because the boundary  $\sigma(z) = \frac{1}{2}$  is a line; (b) fails because affine maps compose to an affine map; (d) is backwards.
- 1F.** Each update adds  $y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \gamma > 0$ . But  $\mathbf{w}^\top \mathbf{w}^*$  could grow simply because  $\|\mathbf{w}\|$  grows without  $\mathbf{w}$  turning toward  $\mathbf{w}^*$ ; the bound  $\|\mathbf{w}_k\| \leq \sqrt{k}R$  shows the length grows only like  $\sqrt{k}$  while the projection grows like  $k$ , so the cosine  $\mathbf{w}_k^\top \mathbf{w}^* / \|\mathbf{w}_k\| \geq \sqrt{k}\gamma/R$ , which cannot exceed 1, forces  $k \leq (R/\gamma)^2$ .
- 2A.** Both give the uniform distribution, so the loss is  $\ln 4 \approx 1.386$  (row 2 of the table).
- 2B.**  $2\sigma(2z) - 1 = \frac{2}{1+e^{-2z}} - 1 = \frac{1-e^{-2z}}{1+e^{-2z}} = \tanh z$ . Differentiating,  $\tanh'(z) = 4\sigma'(2z)$ , so  $\tanh'(0) = 4 \cdot \frac{1}{4} = 1$ .
- 2C.** Using the hint,  $\text{BCE} = \log(1 + e^{-z}) + (1 - y)z$ . Its derivative is  $-\frac{e^{-z}}{1+e^{-z}} + 1 - y = -(1 - \sigma(z)) + 1 - y = \sigma(z) - y$ . For  $z \geq 0$  the stable form is  $z - zy + \log(1 + e^{-z})$ , the same expression. For  $z < 0$  write  $\log(1 + e^{-z}) = -z + \log(1 + e^z)$ ; then  $\text{BCE} = -zy + \log(1 + e^z) = 0 - zy + \log(1 + e^{-|z|})$ .
- 2D.** (b), (d), (e), (g). ReLU and GELU are activations; softmax is an output map.
- 2E.** Halving  $-\ln p_y$  means the new probability is  $\sqrt{p_y}$ . With  $p = e^z/(e^z + S)$ , the odds  $p/(1 - p) = e^z/S$ , so increasing  $z$  by  $\Delta$  multiplies the odds by  $e^\Delta$ :  $\Delta = \ln \frac{\sqrt{p}/(1-\sqrt{p})}{p/(1-p)}$ . For  $p = 0.2$ :  $\sqrt{0.2} = 0.447$ , odds 0.809 vs 0.25, so  $\Delta = \ln 3.24 \approx 1.17$  (the script checks that the loss indeed halves).
- 2F.** The softmax makes the tags compete: probabilities sum to 1, so an image with four true tags can give each at most about 0.25, and thresholds become meaningless. Use 20 sigmoid outputs with a binary cross-entropy per tag (BCEWithLogitsLoss on 20 logits).



# Bibliography

- [Cy89] GEORGE CYBENKO. Approximation by Superpositions of a Sigmoidal Function. In: *Mathematics of Control, Signals and Systems*, **2**: (1989) (cited p. 15)
- [HeGi16] DAN HENDRYCKS and KEVIN GIMPEL. Gaussian Error Linear Units (GELUs). In: *arXiv preprint arXiv:1606.08415*, (2016) (cited p. 21)
- [Li+17] TSUNG-YI LIN, PRIYA GOYAL, ROSS GIRSHICK, KAIMING HE, and PIOTR DOLLÁR. “Focal Loss for Dense Object Detection”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017 (cited p. 25)
- [MiPa69] MARVIN MINSKY and SEYMOUR PAPERT. *Perceptrons*. MIT Press, 1969 (cited p. 13)
- [Ro58] FRANK ROSENBLATT. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. In: *Psychological Review*, **65**:6 (1958) (cited p. 10)